

PROMPT IA 04

Vertical Slice E-commerce : Catalog + Pricing + Inventory + Cart

Objectif : produire un lot backend/frontend exécutable, testé, maintenable et conforme aux décisions d'architecture déjà validées.

Langages	Go backend + Next.js frontend
Données	PostgreSQL par service, Redis pour cache/Cart si justifié
Async	Kafka pour événements métier durables, RabbitMQ pour tâches
Contrats	OpenAPI 3.1 + gRPC/Protobuf + Apicurio
Delivery	Tekton -> Harbor -> FluxCD -> Flagger
Sécurité	Keycloak, OPA, OpenBao, Cilium, Istio Ambient

Instruction centrale : implémenter par petits incréments, compiler/tester après chaque étape, corriger immédiatement les erreurs, factoriser les doublons et conserver le repository exécutable à tout moment.

1. Mission

Tu es l'IA responsable du développement du deuxième vertical slice métier de la plateforme e-commerce. Tu dois livrer un code de niveau production pour quatre domaines : Catalog, Pricing, Inventory et Cart, ainsi que leur intégration avec Product, Search/OpenSearch et le frontend Next.js existants.

Le résultat final doit permettre à un utilisateur de rechercher un produit, ouvrir sa fiche, voir un prix effectif et une disponibilité indicative, sélectionner un SKU/une quantité et ajouter l'article au panier. Aucune réservation de stock définitive n'est faite à ce stade ; elle appartiendra au checkout/Saga.

2. Règles d'architecture non négociables

- 1 microservice = 1 base PostgreSQL logique dédiée, ownership strict, aucune jointure cross-service.
- REST/JSON pour API externe ; gRPC pour appels synchrones service-to-service ; Kafka pour événements métier durables ; RabbitMQ uniquement pour tâches/jobs.
- REST et gRPC appellent exactement les mêmes use cases : aucune duplication de logique métier.
- PostgreSQL est source de vérité ; Redis est cache/état temporaire ; OpenSearch est projection/read model reconstruisible.
- Tout calcul métier final est backend-authoritative. Le frontend/BFF ne décide jamais du prix, du stock ou de la validité métier.
- Idempotence obligatoire pour opérations critiques et consommateurs Kafka.
- Outbox Pattern pour toute publication Kafka issue d'une transaction métier.
- UTC partout ; UUIDv7 pour les nouvelles entités ; aucun float pour les montants monétaires.
- Secrets et credentials récupérés via OpenBao, jamais hardcodés ni stockés durablement dans Git.
- Observabilité OpenTelemetry : traces, métriques, logs structurés, correlation_id et trace_id.
- Code propre, fonctions courtes, factorisation des doublons, interfaces limitées aux frontières nécessaires.

3. Responsabilités métier

Service	Responsabilité	Ne doit pas posséder
Catalog	Catégories, collections, navigation, merchandising, ordre d'affichage, visibilité commerciale, règles de publication.	Fiche produit, stock, calcul du prix effectif.
Pricing	Prix effectifs, promotions, remises, coupons, règles commerciales, périodes de validité, version de pricing.	Stock, taxes, données descriptives produit.
Inventory	Quantités, disponibilité, réservations futures, mouvements de stock, source du stock disponible.	Prix, catalogue, commande.

Service	Responsabilité	Ne doit pas posséder
Cart	Panier utilisateur, SKU, quantité, prix indicatif recalculable et références nécessaires.	Prix final, taxation finale, réservation de stock définitive.

4. Parcours fonctionnel à livrer

```

Product
|
+--> Catalog
+--> Pricing
+--> Inventory
|
v
Search projection enrichie (OpenSearch)
|
v
Next.js / BFF
|
v
Cart

```

- Créer/associer un produit à une catégorie/collection dans Catalog.
- Définir un prix de base puis un prix effectif via Pricing.
- Initialiser et mettre à jour le stock d'un SKU dans Inventory.
- Publier les événements correspondants dans Kafka via Outbox.
- Mettre à jour la projection Search/OpenSearch avec category, price, availability et image.
- Afficher le produit enrichi côté Next.js.
- Ajouter un SKU/quantité au Cart avec prix indicatif et disponibilité vérifiée au moment de l'ajout.
- Lire/modifier/supprimer les lignes du panier.
- Recalculer le panier à la lecture si les versions de pricing/produit/stock ont changé.

5. Contrats API attendus

Définir les contrats exacts dans OpenAPI 3.1 et Protobuf. Les noms ci-dessous sont une base ; garde une convention cohérente dans tout le repository.

Domaine	REST public / BFF	gRPC interne
Catalog	GET /catalog/categories POST /catalog/categories PUT /catalog/categories/{id} POST /catalog/assignments	CatalogService.GetCategory CatalogService.ListCategories CatalogService.AssignProduct
Pricing	GET /pricing/products/{productId} POST /pricing/rules POST /pricing/calculate	PricingService.CalculatePrice PricingService.GetEffectivePrice
Inventory	GET /inventory/skus/{skuld} POST /inventory/adjustments	InventoryService.GetAvailability InventoryService.AdjustStock
Cart	GET /cart POST /cart/items PATCH /cart/items/{id} DELETE /cart/items/{id}	CartService.GetCart CartService.AddItem CartService.UpdateItem CartService.RemoveItem

6. Modèles métier minimum

Catalog

```
Category {  
  id UUIDv7  
  slug string  
  name string  
  parent_id *UUID  
  status ACTIVE|INACTIVE  
  sort_order int  
  created_at UTC  
  updated_at UTC  
}
```

```
ProductAssignment { category_id, product_id, visibility, sort_order }
```

Pricing

```
PriceRule {  
  id UUIDv7  
  product_id / sku_id  
  currency ISO4217  
  amount_minor int64  
  discount_minor int64  
  valid_from UTC  
  valid_to UTC  
  priority int  
  version int64  
}
```

```
Money { amount_minor int64, currency string }
```

Inventory

```
StockItem {  
  sku_id UUIDv7  
  on_hand int64  
  reserved int64  
  available int64  
  version int64  
}
```

```
StockMovement { id, sku_id, delta, reason, idempotency_key, occurred_at }
```

Cart

```
Cart {  
  id UUIDv7  
  customer_id UUIDv7  
  status ACTIVE|CHECKOUT_STARTED|ABANDONED  
  version int64  
}
```

```
CartItem {  
  id UUIDv7  
  sku_id UUIDv7  
  quantity int64  
  indicative_unit_price_minor int64  
  currency string  
  pricing_version int64  
  product_version int64 optional  
}
```

7. Événements Kafka

Tous les événements utilisent l'enveloppe standard validée : event_id, event_type versionné, aggregate_id, aggregate_version, occurred_at, producer, correlation_id, causation_id, trace_id.
Partition key = aggregate_id.

event_type	producer	aggregate_id
catalog.category.created.v1	catalog	category_id
catalog.product.assigned.v1	catalog	product_id
pricing.price.changed.v1	pricing	product_id ou sku_id
inventory.stock.changed.v1	inventory	sku_id
cart.item.added.v1	cart	cart_id
cart.item.updated.v1	cart	cart_id
cart.item.removed.v1	cart	cart_id

Chaque consumer Search doit être idempotent et stocker au minimum event_id / aggregate_version pour ignorer doublons et événements obsolètes.

8. Projection Search/OpenSearch enrichie

Ne jamais faire de N+1 Product/Pricing/Inventory au moment d'afficher une liste. Search maintient un document enrichi alimenté par Kafka.

```
{
  "product_id": "...",
  "sku_id": "...",
  "name": "...",
  "slug": "...",
  "category_ids": [...],
  "effective_price_minor": 9990,
  "currency": "EUR",
  "available": true,
  "available_quantity_hint": 12,
  "image": "...",
  "product_version": 5,
  "pricing_version": 9,
  "inventory_version": 21
}
```

- Mappings explicites ; dynamic mapping libre interdit.
- Index versionné + alias, ex. products-read-v1 -> alias products-read.
- Rebuild complet possible depuis sources durables/Kafka.
- Les valeurs stock/prix de la projection sont indicatives pour affichage ; checkout revalidera toujours Pricing/Inventory.

9. Cart : règles spécifiques

- Le panier conserve un prix indicatif, jamais le prix final contractuel.
- L'ajout vérifie que le SKU existe, que le produit est publiable et qu'une disponibilité indicative est positive.
- Aucune réservation définitive de stock lors de AddItem.
- Lors de GET /cart, recalculer ou signaler les écarts de pricing/availability si nécessaire.

- POST/PATCH doivent être idempotents lorsque l'opération peut être rejouée.
- Quantité strictement positive, plafond raisonnable par ligne, validation côté backend uniquement.
- Prévoir une expiration/abandon du panier sans utiliser Redis comme unique source de vérité si le panier doit être durable.

10. Performance et résilience

- Paralléliser les appels gRPC indépendants avec context deadlines.
- Timeout obligatoire ; retry uniquement sur opérations idempotentes ; backoff + jitter ; aucun retry storm.
- Circuit breaker/bulkhead via politiques applicatives/Istio selon le besoin.
- Redis pour hot reads ciblés seulement après mesure ; PostgreSQL reste source de vérité.
- Cursor pagination pour listes volumineuses.
- Payloads minimaux ; jamais SELECT * sur les chemins critiques.
- Freshness budget explicite : projection Search légèrement stale autorisée, données transactionnelles revalidées.

11. Structure du repository

```
services/
catalog/
pricing/
inventory/
cart/
search/
web/
customer/
contracts/
proto/
openapi/
platform/
deploy/
tekton/
flux/
policies/
```

Chaque service Go :

```
cmd/
internal/domain/
internal/application/
internal/infrastructure/
internal/transport/
api/
migrations/
```

Bibliothèques communes minimales uniquement : observabilité, enveloppe d'événement, erreurs standardisées, helpers techniques réellement partagés. Ne jamais créer un gros framework interne métier.

12. Tests obligatoires

Niveau	Exigence
Unit	Domain rules, calcul Pricing, quantités Inventory, logique Cart.
Integration	PostgreSQL réel via testcontainers-go, migrations, repositories, outbox.
Contract	REST/OpenAPI, gRPC/Buf, Kafka/Protobuf/Apicurio.
Kafka	Idempotence, doublons, événements hors ordre, retry/DLQ.
Search	Projection enrichie, rebuild, alias/index versionné.
E2E	Product -> Catalog/Pricing/Inventory -> Search -> Next.js -> Add Cart.
Race	go test -race ./...
Perf smoke	k6 ciblé sur browse/search/cart.

13. CI/CD et qualité

- gofmt, go vet, golangci-lint, tests unitaires/intégration, race detector, build.
- Trivy, Syft/SBOM, Cosign avant publication Harbor.
- Images distroless, non-root, read-only root filesystem, capabilities drop ALL.
- FluxCD pour déploiement ; Flagger pour rollout progressif des services critiques.
- Aucun latest ; images référencées par digest SHA256 en production.
- Kyverno bloque les manifests non conformes ; Cilium default-deny ; Istio Ambient pour mTLS/identity.
- Aucun secret dans Git ni dans une ConfigMap.

14. Ordre d'implémentation obligatoire

1. Créer contrats OpenAPI/Protobuf et modèles de données.
2. Implémenter Catalog + DB + tests.
3. Implémenter Pricing + calculs monétaires + tests.
4. Implémenter Inventory + optimistic concurrency/versioning + tests.
5. Implémenter Outbox et événements Kafka pour les trois services.
6. Étendre Search consumer et projection OpenSearch enrichie.
7. Implémenter Cart + persistance + validations backend.
8. Intégrer Next.js BFF/RSC : catalogue, fiche produit, panier.
9. Ajouter observabilité complète et dashboards de base.
10. Ajouter E2E + k6 smoke.
11. Finaliser Docker/Tekton/Harbor/FluxCD/Flagger.
12. Faire une passe de refactorisation : supprimer duplication, réduire complexité, documenter décisions.

15. Definition of Done

- ☑ Un produit peut être assigné à un catalogue/catégorie.
- ☑ Un prix effectif peut être calculé avec Money sans float.
- ☑ Un stock par SKU peut être initialisé et ajusté idempotemment.
- ☑ Les changements publient des événements via Outbox/Kafka.
- ☑ Search construit un document OpenSearch enrichi avec catégorie, prix et disponibilité.
- ☑ Next.js affiche les données enrichies sans cascade N+1.
- ☑ Un utilisateur authentifié peut ajouter, modifier, supprimer et relire des lignes de Cart.
- ☑ Cart conserve un prix indicatif mais aucune décision finale de checkout.
- ☑ Tous les services compilent et tous les tests passent.
- ☑ Aucune race détectée.
- ☑ OpenTelemetry relie les appels REST/gRPC/Kafka par trace/correlation IDs.
- ☑ Les images passent scan/SBOM/signature et sont déployables via FluxCD.
- ☑ Le repository reste propre : pas de code mort, pas de duplication inutile, pas de TODO bloquant.

16. Format de travail attendu de l'IA

Ne génère pas tout le projet en une seule réponse aveugle. Travaille par incréments et garde le repository vert.

- Avant chaque étape : annoncer brièvement l'objectif et les fichiers modifiés.
- Après chaque étape : compiler et exécuter les tests pertinents.
- En cas d'échec : corriger avant de continuer.
- Factoriser les répétitions dans des fonctions/packages minimaux sans sur-abstraction.
- Ne jamais masquer une erreur par un mock irréaliste ou par la suppression d'un test.
- Documenter les décisions importantes sous forme d'ADR lorsque nécessaire.
- À la fin : fournir un résumé des fonctionnalités livrées, tests passés, limites connues et prochaines étapes.

Résultat attendu : un vertical slice réellement exécutable où Catalog, Pricing, Inventory et Cart sont des domaines autonomes, observables et testés, tandis que Search/OpenSearch et Next.js exploitent des projections rapides sans devenir sources de vérité métier.